Local Supermemory

Entwicklerhandbuch v1.0

granaria

Februar 2026

Inhaltsverzeichnis

1 Übersicht

Local Supermemory ist ein MCP-Server, der Claude Desktop ein persistentes Gedächtnis gibt. Alle Daten bleiben lokal.

1.1 Architektur

```
Claude Desktop   MCP Server    (stdio)




        ChromaDB        SQLite          Ollama
       (Vektoren)     (Metadaten)      (Profil)
```

Komponenten:

- **ChromaDB** – Vektordatenbank für semantische Ähnlichkeitssuche
- **SQLite** – Relationale Daten: Memories, Projekte, Profil-Cache
- **Ollama** – Lokales LLM für Profil-Generierung (optional)

1.2 Datenspeicherung

```
~/.local-supermemory/
+-- chroma/            # ChromaDB Vektoren
|   +-- chroma.sqlite3
+-- memories.db        # SQLite Metadaten
```

2 Installation

```
git clone https://github.com/granaria/local-supermemory.git
cd local-supermemory
pip install -e .
```

Abhängigkeiten: `mcp`, `chromadb`, `httpx`

2.1 Claude Desktop Config

Datei: `~/Library/Application Support/Claude/claude_desktop_config.json`

```
{
  "mcpServers": {
    "local-supermemory": {
      "command": "python3",
      "args": ["-m", "local_supermemory.server"]
    }
  }
}
```

3 MCP Tools

3.1 memory

Speichert oder löscht eine Memory.

Parameter	Typ	Default	Beschreibung
`action`	string	ßave"	ßave" oder "forget"
`content`	string	–	Memory-Text (max 200KB)
`project`	string	"default"	Projekt-Scope

Interner Ablauf:

1. Content-Hash (SHA256[:16]) als ID generieren
2. ChromaDB: Dokument mit Embedding speichern
3. SQLite: Metadaten persistieren
4. Profil-Cache invalidieren

Listing 1: store.py – save()

```python
def save(self, content: str, project: str = "default"):
    mid = self._gen_id(content)  # SHA256[:16]
    coll = self._get_collection(project)

    # Upsert in ChromaDB
    existing = coll.get(ids=[mid])
    if existing['ids']:
        coll.update(ids=[mid], documents=[content])
    else:
        coll.add(ids=[mid], documents=[content])

    # SQLite: INSERT ... ON CONFLICT DO UPDATE
    # Cache invalidieren
    conn.execute("DELETE FROM profile_cache WHERE project=?")
```

3.2 recall

Semantische Suche mit optionalem Profil.

Parameter	Typ	Default	Beschreibung
`query`	string	–	Suchanfrage
`project`	string	"default"	Projekt-Scope
`include_profile`	bool	true	Profil anhängen
`n_results`	int	15	Max. Ergebnisse

Ablauf:

1. Query embedden (MiniLM-L6-v2, 384d Vektoren)
2. Cosine-Similarity-Suche in ChromaDB Collection
3. Optional: Profil aus Cache oder via Ollama generieren
4. Ergebnisse mit Similarity-Score (0–100%) zurückgeben

Listing 2: store.py – recall()

```python
def recall(self, query: str, project: str = "default", n: int = 15):
    coll = self._get_collection(project)
    res = coll.query(query_texts=[query], n_results=n)

    memories = []
    for i, doc in enumerate(res['documents'][0]):
        dist = res['distances'][0][i]  # Cosine: [0,2]
        memories.append({
            "content": doc,
            "similarity": round((1 - dist) * 100),  # -> %
            "id": res['ids'][0][i]
        })
    return memories
```

3.3 Weitere Tools

list_projects Listet alle Projekte mit Memory-Count

stats Statistiken: Gesamtzahl, Projekte, Pfad

whoami Generiert Profil aus letzten 10 Memories via Ollama

4 Datenbank-Schema

Listing 3: SQLite Schema

```sql
CREATE TABLE memories (
    id TEXT PRIMARY KEY,          -- SHA256[:16] vom Content
    content TEXT NOT NULL,
    project TEXT DEFAULT 'default',
    created_at TEXT NOT NULL,     -- ISO 8601
    updated_at TEXT NOT NULL,
    metadata TEXT                 -- JSON
);

CREATE TABLE projects (
    name TEXT PRIMARY KEY,
    description TEXT,
    created_at TEXT NOT NULL
);

CREATE TABLE profile_cache (
    project TEXT PRIMARY KEY,
    profile TEXT NOT NULL,        -- LLM-generierter Text
    generated_at TEXT NOT NULL
);

CREATE INDEX idx_memories_project ON memories(project);
```

Design-Entscheidungen:

- **Content-Hash als ID** – Deduplizierung: gleicher Text = gleiche ID
- **Profil-Cache** – Vermeidet LLM-Aufrufe, invalidiert bei Änderung
- **Dual Storage** – ChromaDB für Vektoren, SQLite für Queries

5 Profil-Engine

Aggregiert Memories zu strukturiertem Benutzerprofil via Ollama.

Listing 4: profile.py

```
PROMPT = """Analysiere folgende Benutzer-Memories...
Erstelle Profil mit: Identitaet, Hardware/Software,
Projekte, Praeferenzen, Kontakte.
Nur vorhandene Infos, keine Spekulationen."""

class ProfileEngine:
    def __init__(self, url="http://localhost:11434",
                 model="qwen2.5:32b"):
        self.url = url
        self.model = model

    async def generate(self, memories: list) -> str:
        text = "\n".join([f"- {m['content']}" for m in memories[:50]])
        r = await client.post(f"{self.url}/api/generate", json={
            "model": self.model,
            "prompt": PROMPT.format(memories=text),
            "options": {"temperature": 0.3}
        })
        return r.json()["response"]
```

Config: Model änderbar in `profile.py`. Fallback ohne Ollama: einfache Liste.

6 ChromaDB Details

Listing 5: ChromaDB Init

```
self.chroma = chromadb.PersistentClient(
    path=str(self.data_dir / "chroma"),
    settings=Settings(anonymized_telemetry=False)
)

def _get_collection(self, project: str = "default"):
    name = f"memories_{project.replace('-', '_')}"
    return self.chroma.get_or_create_collection(
        name=name,
        metadata={"hnsw:space": "cosine"}
    )
```

Embedding: `all-MiniLM-L6-v2` (384d), automatisch bei `add()`
HNSW-Index: Hierarchical Navigable Small World – ANN-Suche, Cosine Distance $\in [0, 2]$

7 MCP Server

Listing 6: server.py Struktur

```
from mcp.server import Server
from mcp.server.stdio import stdio_server
from mcp.types import Tool, TextContent

server = Server("local-supermemory")
store = MemoryStore()
```

```python

@server.list_tools()
async def list_tools() -> list[Tool]:
    return [Tool(name="memory", inputSchema={...}), ...]

@server.call_tool()
async def call_tool(name: str, args: dict) -> list[TextContent]:
    if name == "memory":
        return [TextContent(type="text", text="...")]

async def run():
    async with stdio_server() as (r, w):
        await server.run(r, w, server.create_initialization_options())
```

Kommunikation: stdio (Subprocess), JSON-RPC, alle Handler async

8 Erweiterung

8.1 Neues Tool

```python
# 1. In list_tools() registrieren
Tool(name="export", description="Export als JSON", inputSchema={})

# 2. In call_tool() implementieren
elif name == "export":
    mems = store.get_all()
    return [TextContent(type="text", text=json.dumps(mems))]
```

8.2 Custom Embedding

```python
from chromadb.utils import embedding_functions
ef = embedding_functions.SentenceTransformerEmbeddingFunction(
    model_name="paraphrase-multilingual-MiniLM-L12-v2"
)
self.chroma.get_or_create_collection(name=name, embedding_function=ef)
```

9 Troubleshooting

Server startet nicht Python ≥3.10 prüfen, `pip install -e .`

Kein Profil Ollama fehlt: `ollama pull qwen2.5:32b`

Leere Suche Projekt-Name prüfen, Collection leer?

ChromaDB Fehler Reset: `rm -rf ~/.local-supermemory/chroma`

10 Cloud vs. Local

Feature	Cloud	Local
Speicher	Cloudflare	`~/.local-supermemory`
Datenschutz	Cloud	100% lokal
Offline	Nein	Ja
Embedding	Proprietär	MiniLM-L6-v2
Profil-LLM	Cloud-API	Ollama
Kosten	Subscription	Kostenlos